



Deep Learning Foundations

Join the lecture online on your dashboard.

Let's start with a minute of silence.

आचार्यात् पादं आधत्ते पादं शिष्यः स्वमेधया ।
पादं सब्रह्मचारिभ्यः पादं कालक्रमेण च ॥

Meaning:

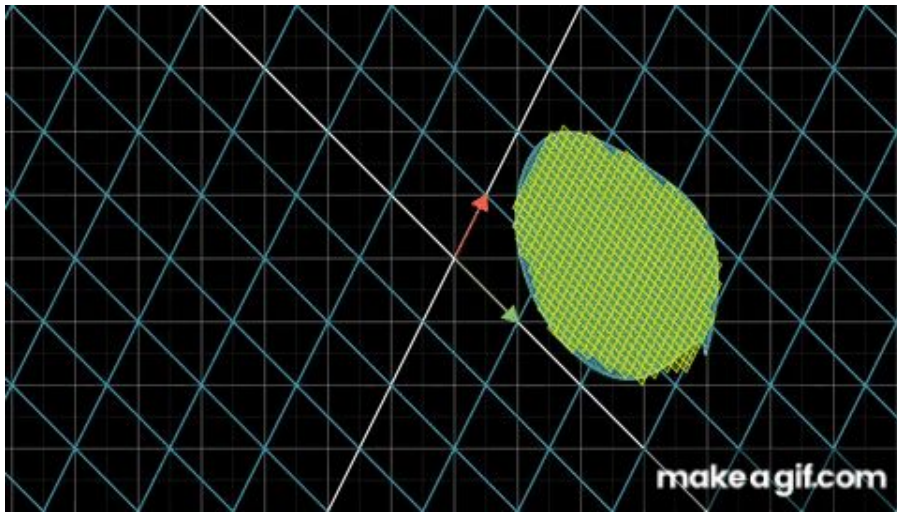
A student acquires knowledge in four equal parts:

- One-fourth from the teacher
- One-fourth through self-reflection and independent thinking
- One-fourth through discussions with peers and fellow learners
- One-fourth over time through personal experience

Concept Recap

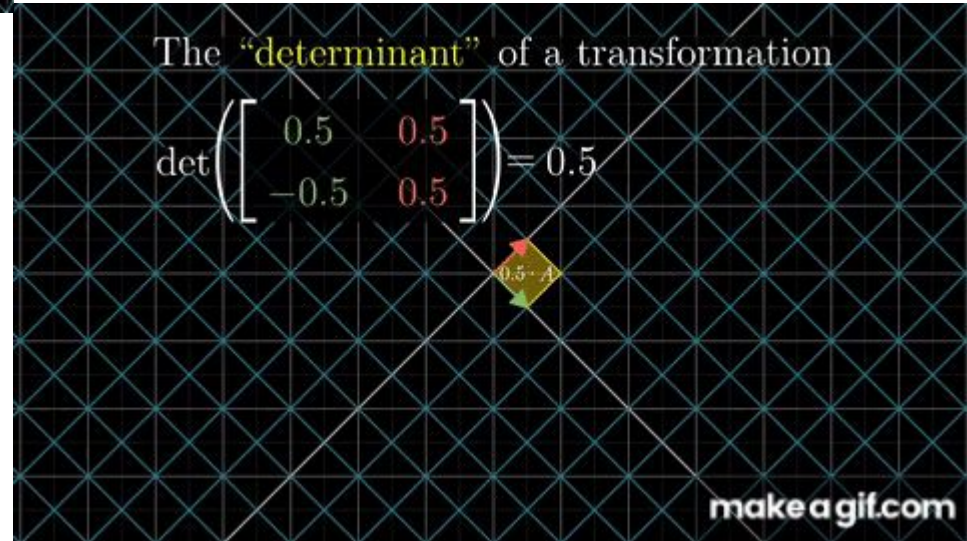
Foundation of Deep Learning:

- Linear Algebra Basics
- Scalars
- Vectors
- Tensors
- Matrices and Operations
- Norms



A matrix with determinant 6

A matrix with 0 determinant

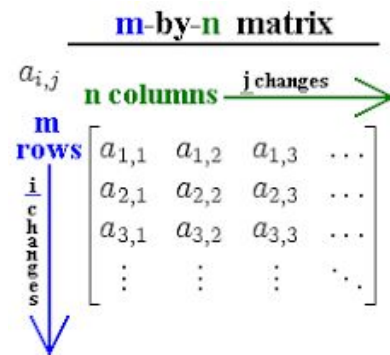


Linear Algebra for Deep Learning – Lecture 2

Matrix Structure • Decompositions • Eigenvalues • SVD • Applications in ML

Will Cover:

- How matrices encode structure
- Why decompositions matter
- How PCA/SVD compress and denoise data
- How linear algebra reveals geometry behind deep learning



Rank

Definition:

The rank of a matrix is the number of linearly independent rows or columns.

Rank = “how much unique information” the matrix contains.

- High rank $\rightarrow$ more information, more dimensions
- Low rank $\rightarrow$ redundancy, compression possible

ML Interpretation

- Rank tells you how many features matter.
- Many real ML datasets are low-rank due to correlations.
- SVD/PCA use this fact for dimensionality reduction.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Rank(A) = 4

Nullity(A) = 2

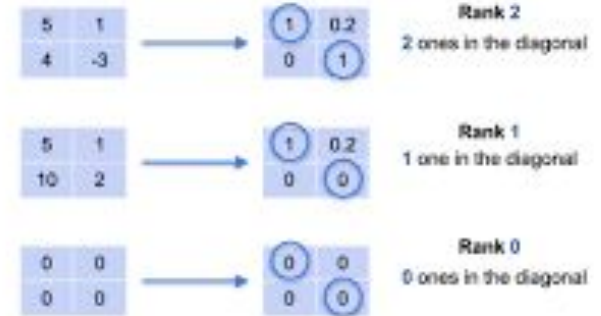
Rank

Example

$$\text{Matrix } A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$$

Row 2 = 2 × Row 1 → rank = 1

Only one direction of information.



Practice Question

If a matrix is 100×100 but rank is only 5, what does that mean?

Column Space, Row Space & Null Space

Column Space ($C(A)$):

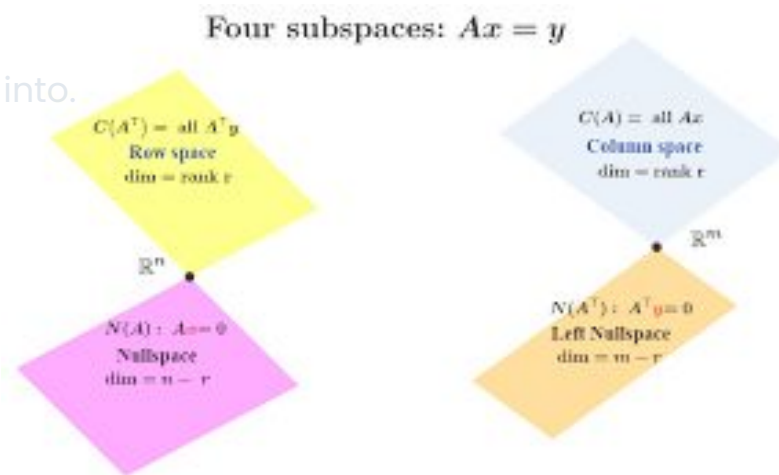
- All possible outputs of Ax .
- Represents the directions a matrix can transform vectors into.

Row Space:

- Describes constraints on x in $Ax = b$.

Null Space ($N(A)$):

- All vectors x such that $Ax = 0$.
- Represents directions the matrix kills (maps to zero).



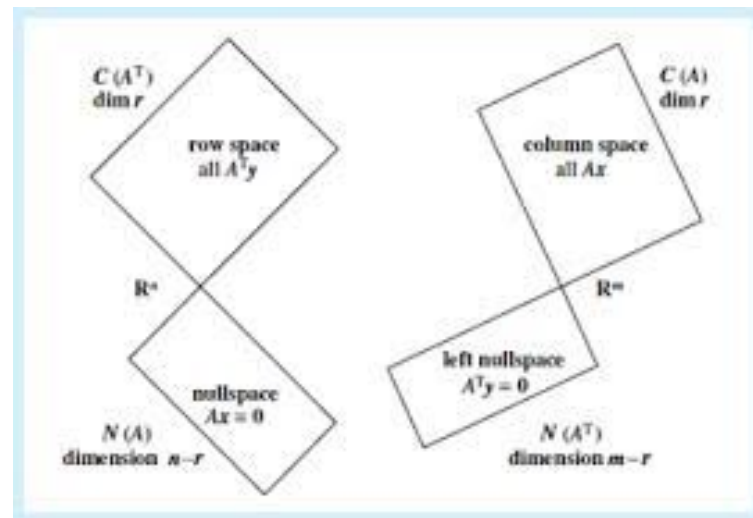
Column Space, Row Space & Null Space

Think of the matrix as a teacher:

- Some concepts it emphasizes → column space
- Some constraints it imposes → row space
- Some mistakes it ignores → null space

Why It Matters in ML

- Null space reveals redundant features
- Column space determines representational capacity
- Row space shows restrictions on outputs
- Used in solving linear systems for optimization



Practice Question

What does a large null space imply about the matrix?

Linear Independence & Basis

Linear Independence

A set of vectors is linearly independent if none of them can be written as a combination of the others.

Why It Matters

ML data often has:

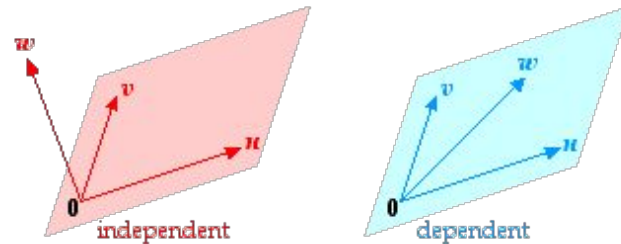
- Redundant features (dependent vectors)
- Only a few true underlying dimensions (independent ones)

Basis

A minimal set of independent vectors that spans a space.

ML Interpretation

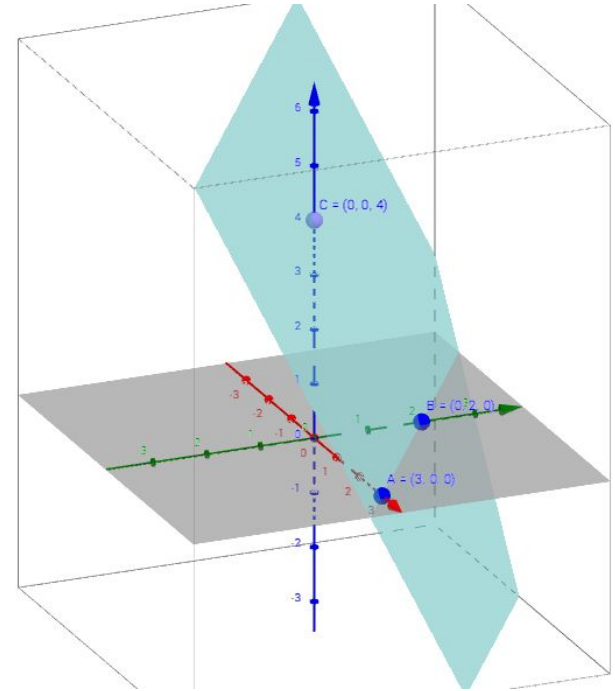
Basis vectors = “core directions” of your data.



Span

The Span of a set of vectors is the set of all possible linear combinations you can create with them.

- Two non-parallel vectors in 3D span a 2D plane.
- Three independent vectors in 3D span the entire space.
- If a vector y is in the span of $\{v_1, \dots, v_n\}$, then y can be reached using those vectors.



Linear Dependence



Linearly Dependent

A set of vectors is dependent if at least one vector can be written as a linear combination of the others. It adds no "new" information or directions to the span.



Linearly Independent

No vector in the set can be formed by combining the others. Every vector provides a unique, necessary direction to define the space.

$$c_1 v_1 + c_2 v_2 + \cdots + c_n v_n = 0$$

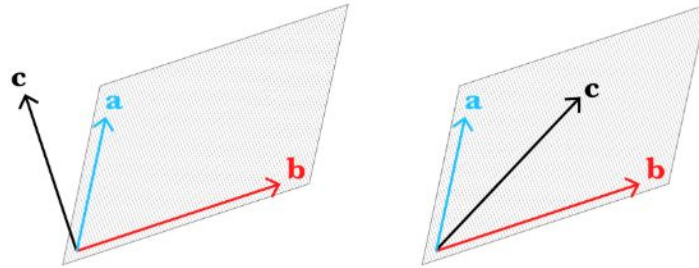
Dependent if there exists a non-trivial solution (not all coefficients are zero).

Visualizing Redundancy

In Machine Learning, linear dependence is synonymous with redundant features.

- If feature A is just double feature B, it doesn't help the model learn anything new—it just takes up memory and can destabilize calculations.

"Independence is information efficiency."



Visualization of three linear independent (left) and dependent (right) vectors in 3-dimensions.

Span and Matrix Rank

Concept	Definition	ML Interpretation
Column Space	The span of the columns of a matrix.	All possible outputs of a model layer.
Rank	The number of linearly independent columns.	The true "intrinsic" dimensionality of data.
Singularity	When columns are dependent (Determinant = 0).	Information loss or redundant layers.

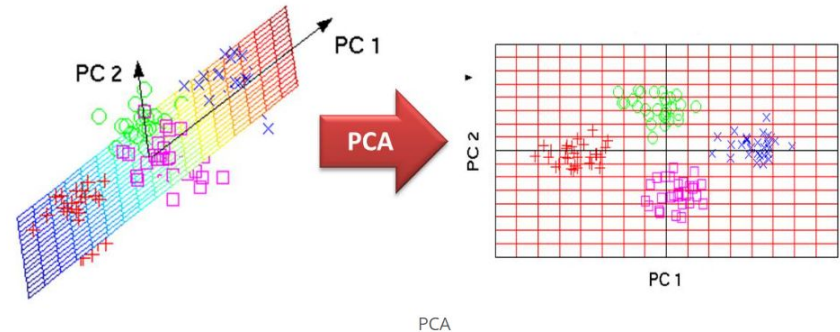
Example:

Principal Component Analysis

PCA uses these concepts to simplify data:

- Identify Span: Find the subspace where data has the most variance.
- Remove Dependence: Discard features that are highly correlated (dependent).
- Efficiency: Lower the dimensionality without losing major information.

Dimensionality Reduction & Principal Component Analysis

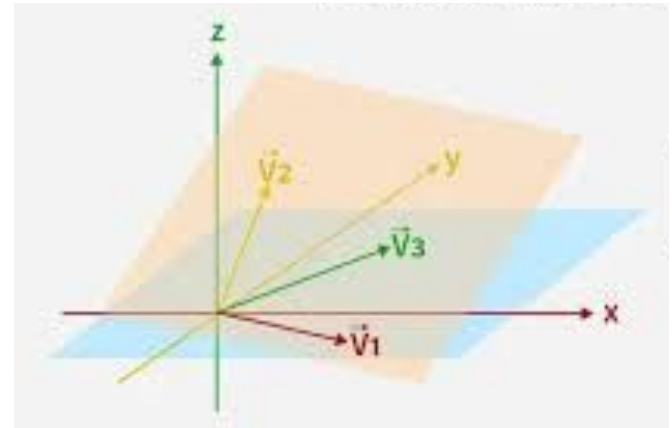


Practice Question

Which of these sets is linearly independent?

A. $\{(1,2), (2,4)\}$

B. $\{(1,2), (2,3)\}$



Orthogonality

Two vectors are orthogonal if:

$$\mathbf{x} \cdot \mathbf{y} = 0$$

Geometric Meaning:

They form a 90° angle $\rightarrow$ no projection of one onto the other.

Why ML Cares:

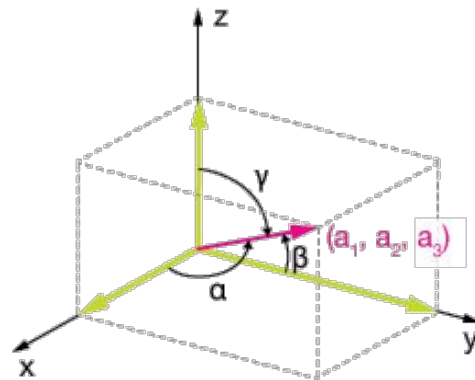
- Orthogonal embeddings reduce feature overlap
- Decorrelated features improve model stability
- Orthogonal weight matrices improve gradient flow
- PCA produces orthogonal principal components

Example

$(1, 2)$ and $(2, -1) \rightarrow$ dot product = 0 $\rightarrow$ orthogonal.

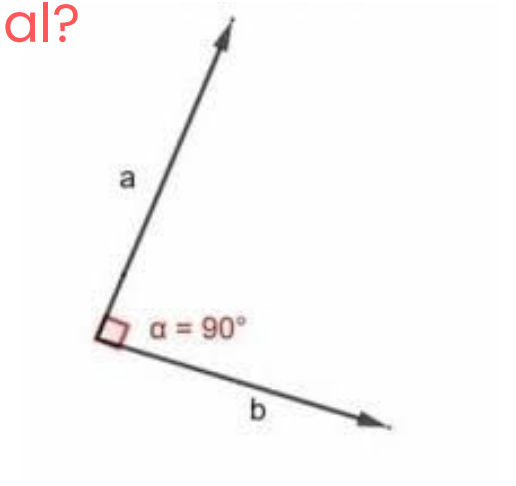
Analogy

Two college friends whose schedules never overlap $\rightarrow$ no interference



Practice Question

If $x = [3, 4]$ and $y = [4, -3]$, are they orthogonal?



Projection Of a Vector

Projection of vector b onto a :

$$\text{proj}_a(b) = (a \cdot b / a \cdot a) a$$

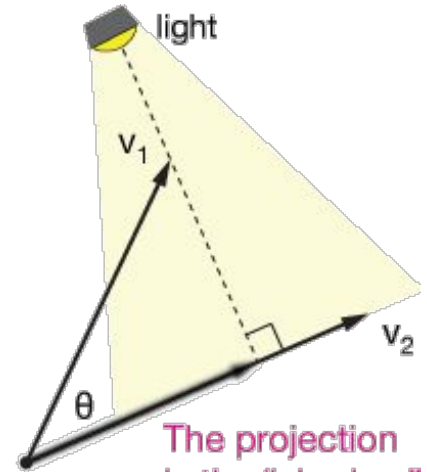
Projection = shadow of one vector onto another direction.

Analogy:

Shining a flashlight on an object $\rightarrow$ shadow cast onto a surface = projection.

When Does Projection Matter?

- When you want to isolate the useful direction
- When removing irrelevant components
- When reducing dimensions



Practice Question

If vector b lies entirely in the null space of a matrix A , what is its projection onto the column space?

Eigenvalues & Eigenvectors

For matrix A:

$$\mathbf{A} \mathbf{v} = \lambda \mathbf{v}$$

Where:

- $\mathbf{v}$ = eigenvector
- λ = eigenvalue

Eigenvectors are directions that A stretches/compresses.

- Eigenvalues are how much it stretches/compresses.



The diagram shows the equation $\mathbf{A} \mathbf{v} = \lambda \mathbf{v}$ with labels below each term. Below $\mathbf{A}$ is 'n x n Matrix'. Below $\mathbf{v}$ is 'Eigenvector'. Below λ is 'Eigenvalue'. Below the second $\mathbf{v}$ is 'Eigenvector'.

Geometric Meaning

- Matrix transformation $\rightarrow$ one direction stays the same, only its length changes.

ML Interpretation:

Eigenvectors show:

- Main directions of variation in data (PCA)
- Stable directions in optimization
- Important components in embeddings

Every eigen decomposition tells you: The hidden structure of A.

Practice Question

If $A v = 5v$, what does eigenvalue 5 mean?

Eigenvalue Intuition

Simplified Story

Imagine a matrix A is a “machine” that transforms vectors:

- For most vectors $\rightarrow$ direction changes
- For special vectors $\rightarrow$ direction stays the same
- Only length changes

These special vectors are eigenvectors

The amount of stretching = eigenvalue

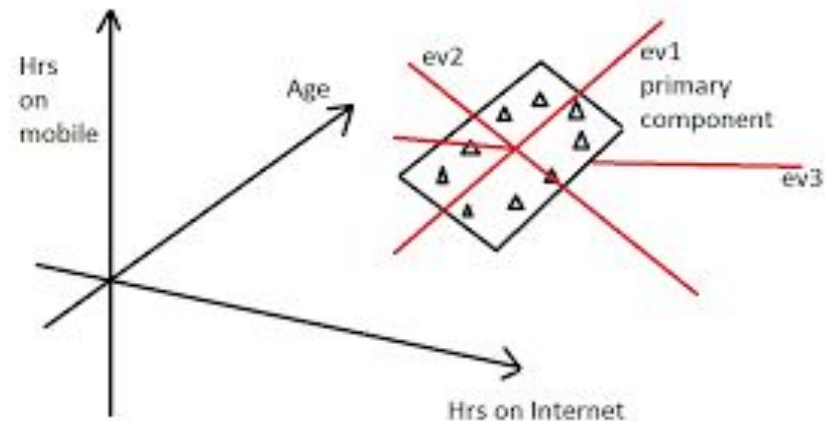
Analogy:

Think of A as a filter:

- It changes most images
- But some images remain in the same “style” after filtering $\rightarrow$ these are eigenvectors

Deep Learning Uses

- PCA $\rightarrow$ find directions with maximum variance
- Spectral clustering $\rightarrow$ use eigenvectors of Laplacian
- Stability in RNNs $\rightarrow$ eigenvalues determine exploding/vanishing behavior



Practice Question

Why do eigenvectors matter in PCA?

Symmetric Matrices

A matrix A is symmetric if:

$$A^T = A$$

Why Symmetric Matrices Matter

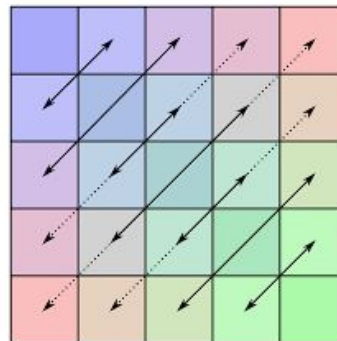
- All eigenvalues are real
- Eigenvectors are orthogonal
- Decomposition is stable & interpretable

ML Applications

- Covariance matrix ($\Sigma = X^T X$) $\rightarrow$ symmetric
- PCA uses covariance matrix
- Many optimization problems use symmetric matrices
- Graph Laplacians in GNNs are symmetric

Symmetric matrices are the nicest matrices to work with:

- Clear geometric interpretation
- Diagonalizable
- Useful for PCA and SVD intuition



Practice Question

Why does PCA always use a symmetric matrix?

Diagonalization

A square matrix $A \in \mathbb{R}^{n \times n}$ is diagonalizable if:

$$A = PDP^{-1}$$

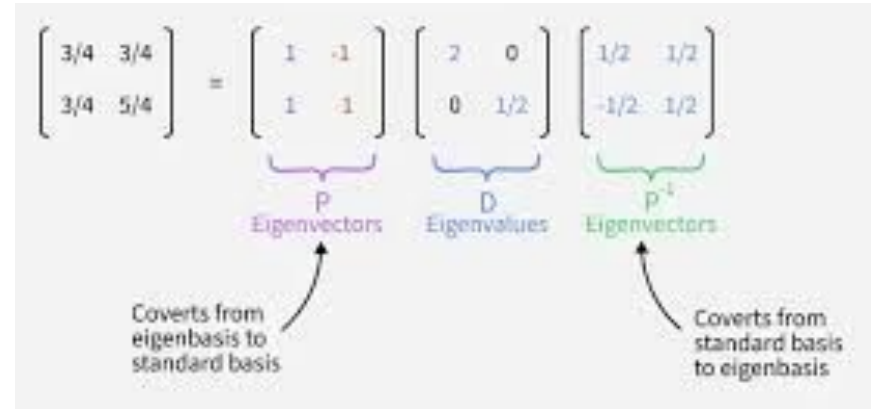
where:

$$P = [v_1 \quad v_2 \quad \cdots \quad v_n]$$

(eigenvectors)

$$D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

(eigenvalues)



$$\begin{bmatrix} 3/4 & 3/4 \\ 3/4 & 5/4 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}}_{\substack{P \\ \text{Eigenvectors}}} \underbrace{\begin{bmatrix} 2 & 0 \\ 0 & 1/2 \end{bmatrix}}_{\substack{D \\ \text{Eigenvalues}}} \underbrace{\begin{bmatrix} 1/2 & 1/2 \\ -1/2 & 1/2 \end{bmatrix}}_{\substack{P^{-1} \\ \text{Eigenvectors}}}$$

Coverts from eigenbasis to standard basis

Coverts from standard basis to eigenbasis

Diagonalization

Mathematical Significance

- Converts a complex matrix into a simple scaling matrix
- Eases computation of: $\mathbf{A}^k$
 - Matrix exponentials $\mathbf{e}^{\mathbf{A}}$
 - Stability analysis in dynamical systems
- Allows easier interpretation of linear transformations

Condition:

A matrix is diagonalizable if it has n linearly independent eigenvectors.

Intuition Behind Diagonalization

Diagonalization finds a new coordinate system where a matrix acts as pure scaling rather than mixing components.

This is expressed through the eigenvector relation:

$$Av_i = \lambda_i v_i$$

This means:

- Each eigenvector v_i points in a direction that the matrix does not rotate or distort
- The transformation only stretches or shrinks that direction by
- The matrix becomes easier to understand in a basis formed by its eigenvectors

Geometric Interpretation

- A matrix normally mixes coordinates (shearing, rotating, stretching)
- Diagonalization “untangles” the transformation into independent dimensions
- In the eigenvector basis:
 - No mixing
 - Only scaling
 - Behavior is predictable

Singular Value Decomposition (SVD)

Formal Definition

For any matrix A:

$$A = U\Sigma V^{\top}$$

Where:

- U: left singular vectors (orthonormal)
- Σ : diagonal matrix of singular values

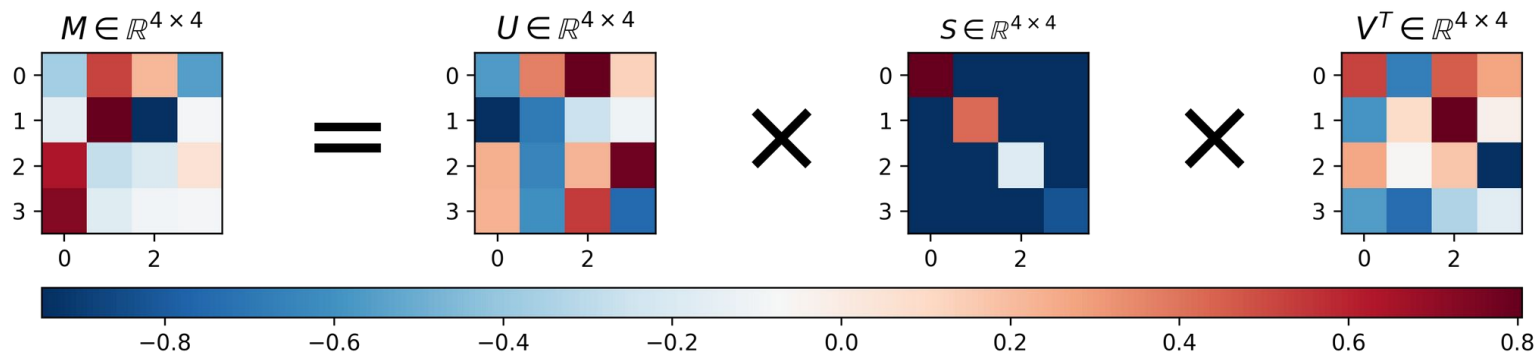
$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots)$$

- V: right singular vectors (orthonormal)

Key Properties

- Works for every matrix (not just square)
- Singular values represent “energy” or importance of directions
- Rank of AAA = number of non-zero singular values
- Generalizes eigen-decomposition

SVD Components $m = 4, n = 4$



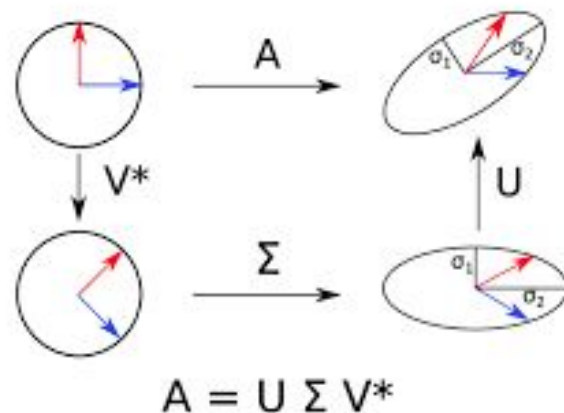
Geometric Interpretation of SVD

Three Transformations

SVD breaks the transformation into:

$$A = U \Sigma V^T$$

- V^T : rotates the input
- Σ : scales (stretches/shrinks)
- U : rotates the output



Geometric Meaning

- Every matrix can be expressed as rotation $\rightarrow$ scaling $\rightarrow$ rotation
- Singular values give axis-aligned scaling
- Orthogonal matrices preserve lengths and angles
- Shows intrinsic geometric structure of data transformation

Geometric Interpretation of SVD

PCA Connection

Apply SVD to centered data matrix X :

$$X = U\Sigma V^T$$

- Columns of V : principal directions
- Singular values give importance of components

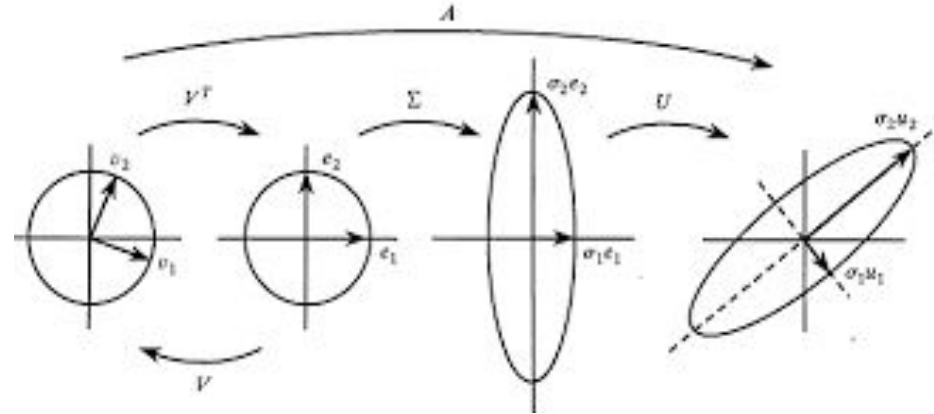
Explained Variance:

The variance captured by component i is:

$$\text{Variance}_i = \sigma_i^2$$

Why PCA Works

- Projects data onto directions of maximum variance
- New axes are orthogonal $\rightarrow$ no redundancy
- Dimensionality reduction retains major structure
- Reduces noise
- Common preprocessing step in ML pipelines



Low-Rank Approximation

Core Idea

Any matrix A can be approximated by keeping only the top- k singular values from its SVD:

$$A \approx A_k = U_k \Sigma_k V_k^\top$$

This gives the best rank- k approximation in terms of minimizing reconstruction error.

Why Low-Rank Works

- Real-world data often has hidden low-dimensional structure
- Noise appears in small singular values
- Keeping major singular values captures essential patterns

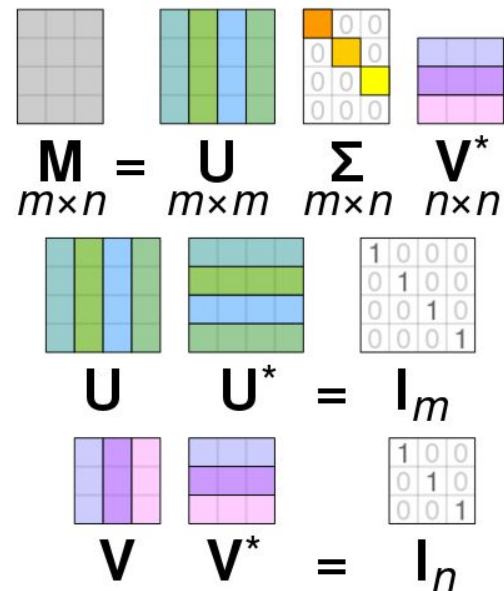
Eckart-Young Theorem

The best rank- k approximation is obtained by truncating SVD:

$$A_k = \arg \min_{\text{rank}(B)=k} \|A - B\|_F$$

ML Relevance

- Compressing large embedding matrices
- Reducing over-parameterized neural layers
- Faster inference using low-rank layers
- Dimensionality reduction before training



Low-Rank Approximation: Why It Works in ML

If SVD shows these singular values:

$$\begin{aligned}\sigma_1 &= 100 \\ \sigma_2 &= 80 \\ \sigma_3 &= 8\end{aligned}$$

- others: <1

Then almost all information is in the first 2–3 directions.

So we approximate:

$$A \approx U_k \Sigma_k V_k^\top$$

Why ML models love this:

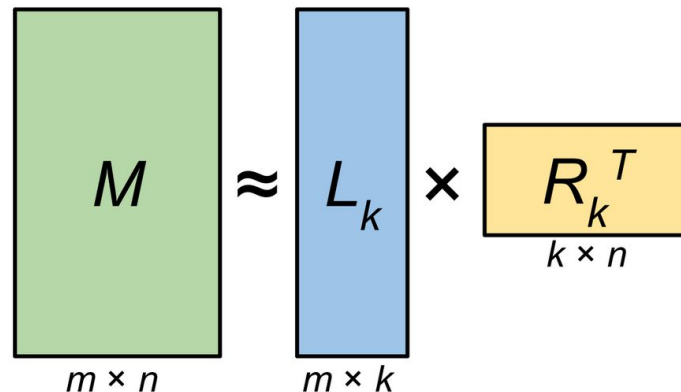
- Remove noise
- Keep essential structure
- Reduce model size
- Make training faster

Analogy:

You can describe a person with:

- height
- weight
- age

You don't need 200 features (hair length, pinky size, etc.)



Covariance Matrix and Principal Directions

Covariance Matrix:

$$C = \frac{1}{n} X^T X$$

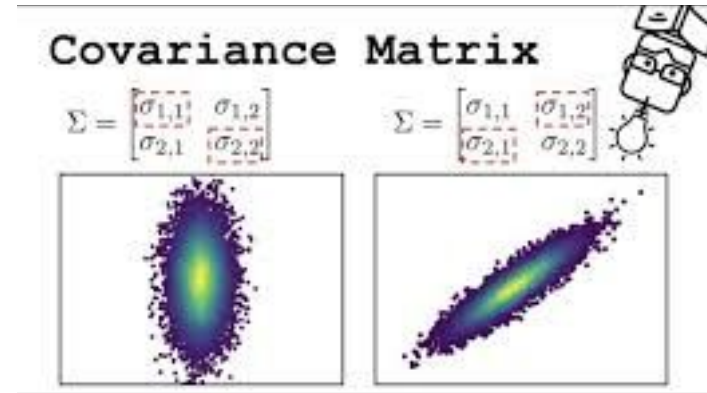
Key properties:

- Symmetric
- Positive semi-definite
- Eigenvectors = principal directions
- Eigenvalues = variances along those directions

Eigen-Decomposition of Covariance

$$C v_i = \lambda_i v_i$$

- v_i : principal components
- λ_i : variance captured by component iii



Covariance Matrix and Principal Directions

Connection to SVD

If $X = U\Sigma V^T$, then:

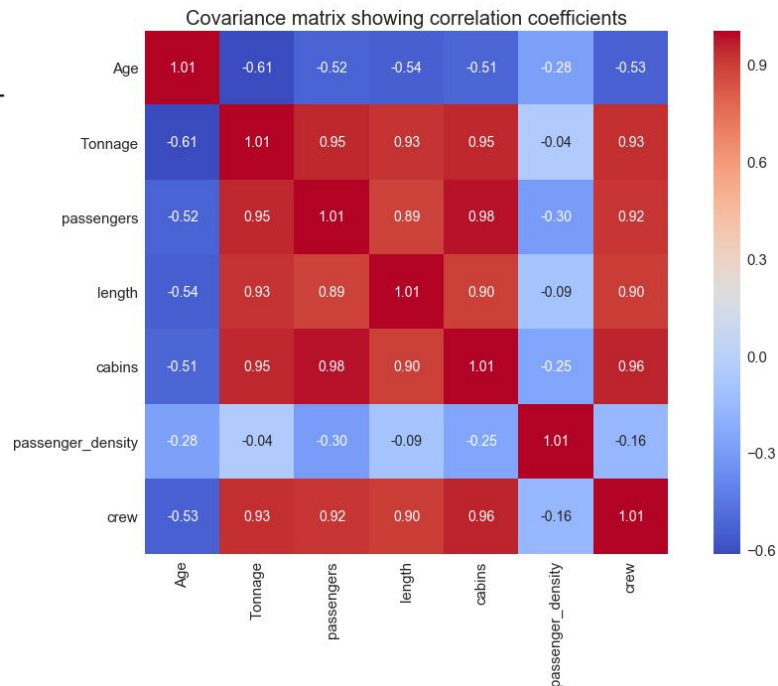
$$C = V\Sigma^2V^T$$

Thus,

- Right singular vectors V = PCA directions
- Singular values give explained variance

ML Usage

- Identify dominant patterns
- Understand correlations
- Preprocessing for many DL models



Matrix Compression Using SVD

Compression Idea

Keep only top-k singular values:

$$A \approx U_k \Sigma_k V_k^\top$$

Storage required reduces from:

$$mn \quad \text{to} \quad k(m + n + 1)$$

where m, n are matrix dimensions.

Where Compression Helps

- Word embedding matrices (Word2Vec, GloVe)
- Vision model feature maps
- Recommender system matrices
- Large linear layers in LLMs

Why Do Models Use SVD/PCA?

If your dataset has:

- 200 columns
- 200 image pixels
- 200 sensor readings

Most of those features are usually saying the same thing.

PCA/SVD help answer:

- “Which features actually matter?”
- “What directions carry the real information?”
- “What can we safely ignore?”

Analogy:

A movie has thousands of frames...

...but the story can be described in a few sentences.

PCA/SVD find those “few sentences” of data.

What SVD Does to Data

Think of your data matrix X as a complicated object.

SVD breaks it into 3 simple steps:

- Rotate the data $\rightarrow V^T$
- Stretch/Shrink the axes $\rightarrow \Sigma$
- Rotate back $\rightarrow U$

Mathematically:

$$X = U\Sigma V^T$$

SVD finds the cleanest, most meaningful directions in your data.

This is exactly how:

- image compression
- noise removal
- topic extraction
- face recognition

actually work under the hood.

	movie 1	movie 2	movie 3	movie 4	movie 5	movie 6	movie 7	movie 8	movie 9	movie 10	...	movie 17770
user 1			1		2							3
user 2		2		3	3			4				
user 3							5	3		4		
user 4	2				3			2				2
user 5		4				5			3			4
user 6			2									
user 7			2					4	2	3		
user 8	3	4				4						
user 9									3			
user 10			1		2							2
...												
user 480189		4			3			3				



reduced-rank
singular
value
decomposition
(sort of)

	movie 1	movie 2	movie 3	movie 4	movie 5	movie 6	movie 7	movie 8	movie 9	movie 10	...	movie 17770
feature 1												
feature 2												
feature 3												
feature 4												
feature 5												

< a bunch of numbers >

+

	feature 1	feature 2	feature 3	feature 4	feature 5
user 1					
user 2					
user 3					
user 4					
user 5					
user 6					
user 7					
user 8					
user 9					
user 10					
...					
user 480189					

< a bunch of numbers >

How PCA Helps You Understand Data

PCA answers:

- “What direction has the highest variance?”
- “Where is the data mostly spread?”
- “What are the strongest patterns?”

PCA uses SVD: $X = U\Sigma V^T$

The columns of V are the principal directions.

Example:

Plotting data in PCA space often reveals:

- hidden clusters
- natural groupings
- outliers
- structure you couldn't see before

Analogy

Like turning a messy object so you can see its shape clearly.

PCA: Step-by-Step Procedure

Step 1 – Center the Data

Given data matrix X :

$$X_{\text{centered}} = X - \mu$$

Each feature has mean zero.

Step 2 – Compute Covariance

This is a symmetric matrix.

$$C = \frac{1}{n} X_{\text{centered}}^{\top} X_{\text{centered}}$$

Step 3 – Perform SVD

Principal directions are columns of V .

$$X_{\text{centered}} = U \Sigma V^{\top}$$

Step 4 – Dimensionality Reduction

Choose top- k components:

$$Z = X_{\text{centered}} V_k$$

This projects data into k -dimensional space.

Real ML Examples

1. Face Recognition (Eigenfaces):

- Faces are represented using PCA directions (“eigenfaces”).
- Only ~50 directions are enough to reconstruct a face.

2. Image Compression (JPEG-like idea)

- Keep only important singular values → smaller file, same image.

3. NLP Embedding Compression

- Large embedding matrices can be approximated using low-rank SVD.

4. Recommender Systems

- User-item matrix → SVD reveals latent interests.

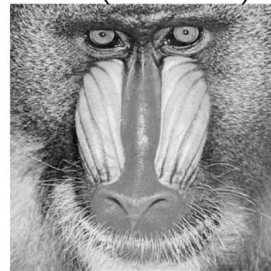
5. Noise Removal

- Drop small singular values to remove noise from signals/images.

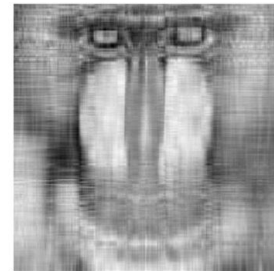
6. Data Visualization

- PCA → convert messy high-dimensional data into 2D/3D shapes.

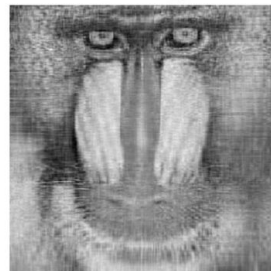
true (rank: 298)



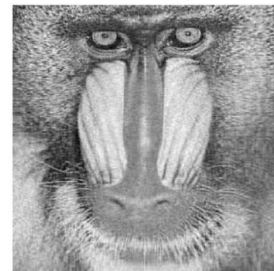
$k = 10$



$k = 20$



$k = 50$



Summary

- Eigenvalues and eigenvectors reveal how a matrix transforms space.
- Diagonalization simplifies a matrix into independent scaling directions.
- SVD breaks any matrix into rotations + scaling and is fundamental in ML.
- PCA uses SVD to find the most informative directions in data.
- Low-rank approximation keeps only the essential structure, reducing noise and size.
- These tools power applications like compression, visualization, and feature reduction.

**Thank you for a great session please fill the
feedback form**